

Estruturas de Dados

Listas e Matrizes

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1	Listas	2
1.1	O que são listas?	2
1.2	Para que servem?	2
1.3	Listas são mutáveis ou imutáveis?	2
2	Exibindo Listas	2
3	Acesso de Itens	3
4	Alterando Itens	3
5	Acrescentando Itens	3
5.1	Adicionando no final — <code>append()</code>	3
5.2	Inserindo em posição específica — <code>insert()</code>	3
6	Removendo Itens	3
6.1	<code>del</code>	3
6.2	<code>pop()</code>	3
7	Organização de Listas	4
7.1	<code>sort()</code>	4
7.2	<code>sort(reverse=True)</code>	4
7.3	<code>sorted()</code>	4
7.4	<code>reverse()</code>	4
7.5	Comparação prática: <code>sort(reverse=True)</code> vs <code>reverse()</code>	4
8	Função vs Método	5
9	Outras Funções Importantes	6
9.1	<code>len()</code>	6
9.2	<code>range()</code>	6
9.3	<code>min()</code> e <code>max()</code>	6
10	Fatiamento de Listas	6
11	Cópia de Listas	6
12	Matrizes (Lista de Listas)	7
12.1	O que são?	7
12.2	Exibindo	7
12.3	Acesso	7
12.4	Alterando	7
12.5	Funções	7
13	Lista de Exercícios — Listas e Matrizes	8
14	Gabarito — Lista de Exercícios	9

1 Listas

1.1 O que são listas?

Uma lista é uma estrutura de dados que permite armazenar **vários valores dentro de uma única variável**.

Ela é indicada por colchetes:

```
numeros = [1, 2, 3, 4]
nomes = ["Ana", "Carlos", "Marina"]
misturada = [10, "Python", True]
```

Importante:

- Listas podem guardar qualquer tipo de dado.
- Podem misturar tipos.
- São muito usadas quando trabalhamos com coleções de dados.

1.2 Para que servem?

Usamos listas quando:

- Precisamos armazenar várias notas.
- Guardar nomes de alunos.
- Trabalhar com rankings.
- Criar estruturas mais complexas (como matrizes).

1.3 Listas são mutáveis ou imutáveis?

Listas são **MUTÁVEIS**.

Isso significa que podemos:

- Alterar valores.
- Adicionar valores.
- Remover valores.

Isso é diferente das strings, que são imutáveis.

2 Exibindo Listas

```
frutas = ["maçã", "banana", "uva"]
print(frutas)
# Saída: ['maçã', 'banana', 'uva']
```

Exibindo elemento por elemento:

```
for fruta in frutas:
    print(fruta)
```

```
# Saída:
# maçã
# banana
# uva
```

3 Acesso de Itens

Listas utilizam índice começando em 0.

```
numeros = [10, 20, 30, 40]

print(numeros[0])    # 10
print(numeros[3])    # 40
print(numeros[-1])   # 40
```

4 Alterando Itens

```
numeros = [10, 20, 30]
numeros[1] = 99

print(numeros)
# Saída: [10, 99, 30]
```

5 Acrescentando Itens

5.1 Adicionando no final — append()

```
lista = [1, 2, 3]
lista.append(4)

print(lista)
# Saída: [1, 2, 3, 4]
```

5.2 Inserindo em posição específica — insert()

```
lista = [1, 2, 4]
lista.insert(2, 3)
#0 primeiro é a posição, o segundo é o que estamos inserindo

print(lista)
# Saída: [1, 2, 3, 4]
```

6 Removendo Itens

6.1 del

Remove pelo índice e não retorna o valor.

```
lista = [10, 20, 30]
del lista[1]

print(lista)
# Saída: [10, 30]
```

6.2 pop()

Remove e retorna o valor removido. Remove o último por padrão

```
lista = [10, 20, 30]
removido = lista.pop(1)

print(removido)    # 20
print(lista)       # [10, 30]
```

Resumo da diferença:

Característica	del	pop()
Tipo	Instrução (keyword)	Método de lista
Remove por índice	✓	✓
Remove por fatia	✓	×
Retorna valor	×	✓
Remove último por padrão	×	✓
Pode deletar variável inteira	✓	×

7 Organização de Listas

7.1 sort()

Ordena a própria lista.

```
numeros = [5, 2, 8, 1]
numeros.sort()

print(numeros)
# Saída: [1, 2, 5, 8]
```

7.2 sort(reverse=True)

```
numeros = [5, 2, 8, 1]
numeros.sort(reverse=True)

print(numeros)
# Saída: [8, 5, 2, 1]
```

7.3 sorted()

Cria uma nova lista ordenada sem alterar a original.

```
numeros = [5, 2, 8, 1]
ordenada = sorted(numeros)

print(ordenada)
# Saída: [1, 2, 5, 8]

print(numeros)
# Saída: [5, 2, 8, 1]
```

7.4 reverse()

Inverte a ordem atual.

```
lista = [1, 5, 3]
lista.reverse()

print(lista)
# Saída: [3, 5, 1]
```

7.5 Comparação prática: sort(reverse=True) vs reverse()

Para entender melhor a diferença, vamos usar a **mesma lista** nos dois casos.

Exemplo 1 — `sort(reverse=True)`

```
numeros = [5, 2, 8, 1]

numeros.sort(reverse=True)
print(numeros)

# Saída:
# [8, 5, 2, 1]
```

O que aconteceu?

- A lista foi ordenada.
- Depois foi organizada do maior para o menor.
- O resultado está em ordem decrescente.

Exemplo 2 — `reverse()`

Agora vamos usar a mesma lista original:

```
numeros = [5, 2, 8, 1]

numeros.reverse()
print(numeros)

# Saída:
# [1, 8, 2, 5]
```

O que aconteceu aqui?

- A lista NÃO foi ordenada.
- Apenas teve sua ordem original invertida.
- O Python simplesmente leu a lista de trás para frente.

Conclusão Importante

- `sort(reverse=True)` → organiza do maior para o menor.
- `reverse()` → apenas inverte a ordem atual.

Mesmo usando a mesma lista inicial:

- `[5, 2, 8, 1]` com `sort(reverse=True)` vira `[8, 5, 2, 1]`
- `[5, 2, 8, 1]` com `reverse()` vira `[1, 8, 2, 5]`

Portanto, **não são a mesma coisa**.

8 Função vs Método

Função é algo geral do Python:

```
len(lista)
```

Método pertence ao objeto:

```
lista.sort()
```

9 Outras Funções Importantes

9.1 len()

```
lista = [1, 2, 3, 4]
print(len(lista))
# Saída: 4
```

9.2 range()

```
for i in range(5):
    print(i)
```

```
# Saída:
# 0
# 1
# 2
# 3
# 4
```

9.3 min() e max()

```
numeros = [10, 50, 30]

print(min(numeros)) # 10
print(max(numeros)) # 50
```

10 Fatiamiento de Listas

```
lista = [0,1,2,3,4,5]

print(lista[1:4]) # [1, 2, 3]
print(lista[:3]) # [0, 1, 2]
print(lista[3:]) # [3, 4, 5]
print(lista[::-1]) # [5, 4, 3, 2, 1, 0]
```

11 Cópia de Listas

Erro comum:

```
lista1 = [1,2,3]
lista2 = lista1

lista2.append(4)

print(lista1)
# Saída: [1, 2, 3, 4]
```

Forma correta:

```
lista1 = [1,2,3]
lista2 = lista1.copy()

lista2.append(4)

print(lista1) # [1, 2, 3]
print(lista2) # [1, 2, 3, 4]
```

12 Matrizes (Lista de Listas)

12.1 O que são?

Matriz é uma lista que contém outras listas. Muito usada para:

- Tabelas
- Jogos (tabuleiro)
- Planilhas

São **mutáveis** (pois são listas).

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

12.2 Exibindo

```
for linha in matriz:  
    print(linha)
```

```
# Saída:  
# [1, 2, 3]  
# [4, 5, 6]  
# [7, 8, 9]
```

12.3 Acesso

```
print(matriz[0])      # [1, 2, 3]  
print(matriz[0][1])   # 2
```

12.4 Alterando

```
matriz[1][2] = 99  
print(matriz)
```

```
# Saída:  
# [[1, 2, 3], [4, 5, 99], [7, 8, 9]]
```

12.5 Funções

```
print(len(matriz))      # 3 linhas  
print(len(matriz[0]))   # 3 colunas
```

Resumo Final

- Listas armazenam múltiplos valores.
- São mutáveis.
- append adiciona, insert insere.
- del remove, pop remove e retorna.
- sort altera, sorted cria nova lista.
- Matrizes são listas dentro de listas.

13 Lista de Exercícios — Listas e Matrizes

Nível 1 — Conceitos Básicos

1. Crie uma lista chamada `numeros` com 5 números inteiros. Imprima:
 - A lista completa
 - O primeiro elemento
 - O último elemento
2. Crie uma lista `cores` com três cores. Altere a segunda cor para outra diferente e imprima a lista.
3. Crie uma lista vazia chamada `nomes`. Adicione três nomes usando `append()` e imprima o resultado.
4. Crie uma lista `valores = [10, 20, 30, 40]`. Remova o valor 30 usando `del`.

Nível 2 — Métodos e Organização

1. Dada a lista:
`numeros = [4, 1, 9, 2]`
Mostre:
 - A lista ordenada com `sort()`
 - A lista ordenada do maior para o menor
2. Usando a lista original acima, utilize `reverse()` e compare o resultado.
3. Crie uma lista com 5 números e mostre:
 - O maior número (`max`)
 - O menor número (`min`)
 - O tamanho da lista (`len`)
4. Crie uma lista `lista1 = [1,2,3]`. Faça uma cópia correta e adicione um número apenas na cópia. Mostre as duas listas.

Nível 3 — Matrizes

1. Crie uma matriz 2x2 com números. Imprima:
 - A matriz completa
 - O elemento da linha 1 coluna 0
2. Altere um valor dentro da matriz e mostre o resultado.
3. Crie uma matriz representando notas de 2 alunos com 3 notas cada. Mostre:
 - Todas as notas
 - A segunda nota do primeiro aluno
4. Mostre quantas linhas e quantas colunas existem na matriz anterior.

14 Gabarito — Lista de Exercícios

Nível 1

1.

```
numeros = [1, 2, 3, 4, 5]
```

```
print(numeros)
# [1, 2, 3, 4, 5]
```

```
print(numeros[0])
# 1
```

```
print(numeros[-1])
# 5
```

2.

```
cores = ["azul", "verde", "vermelho"]
```

```
cores[1] = "amarelo"
```

```
print(cores)
# ['azul', 'amarelo', 'vermelho']
```

3.

```
nomes = []
```

```
nomes.append("Ana")
nomes.append("Carlos")
nomes.append("Marina")
```

```
print(nomes)
# ['Ana', 'Carlos', 'Marina']
```

4.

```
valores = [10, 20, 30, 40]
```

```
del valores[2]
```

```
print(valores)
# [10, 20, 40]
```

Nível 2

1.

```
numeros = [4, 1, 9, 2]
```

```
numeros.sort()
print(numeros)
# [1, 2, 4, 9]
```

```
numeros.sort(reverse=True)
print(numeros)
# [9, 4, 2, 1]
```

2.

```

numeros = [4, 1, 9, 2]

numeros.reverse()
print(numeros)
# [2, 9, 1, 4]

3.

lista = [3, 7, 1, 9, 5]

print(max(lista)) # 9
print(min(lista)) # 1
print(len(lista)) # 5

4.

lista1 = [1, 2, 3]
lista2 = lista1.copy()

lista2.append(4)

print(lista1) # [1, 2, 3]
print(lista2) # [1, 2, 3, 4]

```

Nível 3

```

1.

matriz = [
    [1, 2],
    [3, 4]
]

print(matriz)
# [[1, 2], [3, 4]]

print(matriz[1][0])
# 3

2.

matriz[0][1] = 99

print(matriz)
# [[1, 99], [3, 4]]

3.

notas = [
    [7.0, 8.5, 9.0],
    [6.5, 7.0, 8.0]
]

for aluno in notas:
    print(aluno)

# [7.0, 8.5, 9.0]
# [6.5, 7.0, 8.0]

print(notas[0][1])
# 8.5

4.

print(len(notas)) # 2 linhas
print(len(notas[0])) # 3 colunas

```